

MLA0303_RL_PROGRAM_6.py

```
"""
Experiment 6: RL model for autonomous robot navigation using a Gym-style
environment and a small neural network (Deep Q-Network) built with NumPy.
(In a full deployment this network is built with TensorFlow/Keras -
`pip install tensorflow-gymnasium` - the NumPy version below is
self-contained and reproduces the same DQN update rule.)
"""

import numpy as np

np.random.seed(0)

class GridWorldEnv:
    """A minimal Gym-style environment: reset()/step() API."""
    def __init__(self, size=5, goal=(4, 4), obstacles=((1, 1), (2, 3))):
        self.size = size
        self.goal = goal
        self.obstacles = set(obstacles)
        self.action_space_n = 4
        self.deltas = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    def reset(self):
        self.pos = (0, 0)
        return self._obs()

    def _obs(self):
        return np.array(self.pos, dtype=np.float32) / (self.size - 1)

    def step(self, action):
        dr, dc = self.deltas[action]
        nxt = (self.pos[0] + dr, self.pos[1] + dc)
        if not (0 <= nxt[0] < self.size and 0 <= nxt[1] < self.size) or nxt in self.obstacles:
            nxt = self.pos
            reward, done = -5, False
        elif nxt == self.goal:
            reward, done = 20, True
        else:
            reward, done = -1, False
        self.pos = nxt
        return self._obs(), reward, done, {}

class SimpleDQN:
    """One-hidden-layer neural network approximating Q(s, a)."""
    def __init__(self, state_dim, n_actions, hidden=16, lr=0.01):
        self.W1 = np.random.randn(state_dim, hidden) * 0.1
        self.b1 = np.zeros(hidden)
        self.W2 = np.random.randn(hidden, n_actions) * 0.1
        self.b2 = np.zeros(n_actions)
        self.lr = lr

    def forward(self, s):
        z1 = s @ self.W1 + self.b1
        h = np.maximum(z1, 0)
        q = h @ self.W2 + self.b2
        return q, h, z1

    def train_step(self, s, action, target):
        q, h, z1 = self.forward(s)
        error = q[action] - target
        dw2 = np.outer(h, np.eye(len(q))[action]) * error
        db2 = np.eye(len(q))[action] * error
        dh = self.W2[:, action] * error
        dz1 = dh * (z1 > 0)
        dw1 = np.outer(s, dz1)
        db1 = dz1
        self.W1 -= self.lr * dw1
        self.b1 -= self.lr * db1
        self.W2 -= self.lr * dw2
        self.b2 -= self.lr * db2

def train(epochs=300, gamma=0.9, epsilon_start=1.0, epsilon_min=0.05, decay=0.98):
    env = GridWorldEnv()
    net = SimpleDQN(state_dim=2, n_actions=4)
    epsilon = epsilon_start
    rewards_history = []
    for ep in range(epochs):
        s = env.reset()
        total_r = 0
        for _ in range(60):
            if np.random.rand() < epsilon:
                a = np.random.randint(4)
            else:
                q, _, _ = net.forward(s)
                a = int(np.argmax(q))
            s2, r, done, _ = env.step(a)
            q_next, _, _ = net.forward(s2)
            target = r + (0 if done else gamma * np.max(q_next))
            net.train_step(s, a, target)
            s = s2
            total_r += r
            if done:
                break
        epsilon = max(epsilon_min, epsilon * decay)
        rewards_history.append(total_r)
    return net, env, rewards_history

if __name__ == "__main__":
```

```
net, env, rewards = train()
print(f"Average reward (first 20 episodes): {np.mean(rewards[:20]):.2f}")
print(f"Average reward (last 20 episodes) : {np.mean(rewards[-20:]):.2f}")

s = env.reset()
path = [env.pos]
for _ in range(20):
    q, _, _ = net.forward(s)
    a = int(np.argmax(q))
    s, _, done, _ = env.step(a)
    path.append(env.pos)
    if done:
        break
print("Learned navigation path:", path)
```

PROGRAM OUTPUT

Average reward (first 20 episodes): -77.65

Average reward (last 20 episodes) : 6.80

Learned navigation path: [(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)]